THEORY EXERCISE:-

Q1 Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

History and Evolution of C Programming Language :-

The C programming language was developed in 1972 by Dennis Ritchie at Bell Labs. It was mainly created to develop the Unix operating system. Before C, programming was done in assembly language, which was difficult to use. C made programming easier, faster, and more efficient.

Evolution of c language:-

ALGOL 60 (1960)

BCPL (1967)

B (1970)

"New B" and Initial C (1971-1972)

K&R C (1978)

ANSI C / C89 (1989) & ISO C / C90 (1990)

C99 (1999)

C11 (2011)

C17/C18 (2018)

C23 (2024)

Why C is Still Used Today:-

It is fast and efficient

It provides low-level memory access

It is used in embedded systems and microcontrollers

Many operating systems are written in C

It helps programmers understand computer fundamentals

Q2 Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

1. Install C Compiler (GCC)

Steps:

Using GCC

Go to the official GCC website or download using MinGW

Download MinGW / GCC installer

Run the installer and select:

C Compiler

C++ Compiler (optional but recommended)

Click Install

Add GCC to System PATH

Open Command Prompt

Type:gcc --version

2. Install Visual Studio Code

Steps:

Download VS Code

Install VS Code

Open VS Code

Go to Extensions

Install:

C/C++ Extension (Microsoft)

Install GCC (MinGW) if not installed

Create new .c file

Write code and run

Q3 Explain the basic structure of a C program, including headers, main function,

comments, data types, and variables. Provide examples.

1. Header Files

Header files are used to include library functions in the program.

Example:

```
#include <stdio.h>
```

2. Main Function

Every C program starts from the main() function.

Example:

```
int main()
{
   printf("Hello World");
   return 0;
}
```

3. Comments

Comments are used to explain the program.

Compiler ignores comments.

Single Line Comment

```
// This is a comment
```

4. Data Types

Data types define the type of data stored in variables.

Data Type	Example
int	int a = 10;
float	float b = 5.5;
char	char c = 'A';
double	double d = 12.34;

Example:

```
int a = 10;
float b = 5.5;
char c = 'A';
```

5. Variables

Variables are used to store values.

Example:

```
int a = 10;
int b = 20;
int sum;

sum = a + b;
```

Complete Example Program:

```
#include <stdio.h>
```

```
// Program to add two numbers

int main()
{
   int a = 10;
   int b = 20;
   int sum;

   sum = a + b;

   printf("Sum = %d", sum);

   return 0;
}
```

Q4 Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators

1. Arithmetic Operators

Arithmetic operators are used for mathematical calculations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Example:

```
#include <stdio.h>
int main()
{
   int a = 10, b = 5;

   printf("Addition = %d\n", a + b);
   printf("Subtraction = %d\n", a - b);
   printf("Multiplication = %d\n", a * b);
   printf("Division = %d\n", a / b);
   printf("Modulus = %d\n", a % b);

   return 0;
}
```

2. Relational Operators

Relational operators compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than equal to
<=	Less than equal to

Example:

```
int a = 10, b = 20;
```

```
printf("%d\n", a > b);
printf("%d\n", a < b);
```

3. Logical Operators

Logical operators are used to combine conditions.

Operator	Meaning
&&	AND
!	NOT

Example:

```
int a = 10, b = 20;

if(a < b && a != 0)
{
   printf("Condition True");
}
```

4. Assignment Operators

Assignment operators assign values to variables.

Operator	Example	Meaning
=	a = 10	Assign
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5

Example:

```
int a = 10;

a += 5;
printf("%d", a);
```

5. Increment and Decrement Operators

Used to increase or decrease value by 1.

Operator	Meaning
++	Increment
--	Decrement

Example:

```
int a = 10;

a++;
printf("%d\n", a);

a--;
printf("%d", a);
```

6. Bitwise Operators

Bitwise operators work on binary numbers.

Operator	Meaning

&	AND
^	XOR
~	NOT
<<	Left shift
>>	Right shift

Example:

```
int a = 5, b = 3;

printf("%d", a & b);
```

7. Conditional (Ternary) Operator

Conditional operator is used as short form of if-else.

Syntax:

```
condition ? true : false;
```

Example:

```
int a = 10, b = 20;

int max = (a > b) ? a : b;

printf("Maximum = %d", max);
```

Q5 Explain decision-making statements in C (if, else, nested if-else, switch).

Provide examples of each.

Decision-Making Statements in C Programming Language

Decision-making statements are used to control the flow of program based on conditions.

Types of Decision-Making Statements:

if

if-else

nested if-else

switch

1. if Statement

The if statement is used to execute code when condition is true.

Syntax

```
if(condition)
{
  statements;
}
```

Example

```
#include <stdio.h>
int main()
{
   int a = 10;

   if(a > 5)
   {
      printf("A is greater than 5");
```

```
    }

    return 0;
}
```

2. if-else Statement

The if-else statement executes one block if condition is true and another if false.

Syntax

```
if(condition)
{
  statements;
}
else
{
  statements;
}
```

Example

```
#include <stdio.h>
int main()
{
    int num = 7;

    if(num % 2 == 0)
    {
        printf("Even Number");
    }
    else
    {
        printf("Odd Number");
    }
```

```
    return 0;
}
```

3. Nested if-else

Nested if-else means if statement inside another if statement.

Example

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20;

    if(a > b)
    {
        printf("A is greater");
    }
    else if(b > a)
    {
        printf("B is greater");
    }
    else
    {
        printf("Both are equal");
    }

    return 0;
}
```

4. switch Statement

The switch statement is used when there are multiple choices.

Syntax

```
switch(expression)
{
  case value1:
     statements;
     break;

  case value2:
     statements;
     break;

  default:
     statements;
}
```

Example

```
#include <stdio.h>
int main()
{
   int day = 2;

   switch(day)
   {
      case 1:
         printf("Monday");
         break;

      case 2:
         printf("Tuesday");
         break;
```

```
        case 3:
            printf("Wednesday");
            break;

        default:
            printf("Invalid Day");
    }

    return 0;
}
```

Q6 Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

Loops in C Programming Language

Loops are used to repeat statements multiple times.

Types of loops:

while

for

do-while

1. while Loop

Checks condition before execution.

Syntax

while(condition)

```
{
  statements;
}
```

Example

```
int i = 1;
while(i <= 5)
{
  printf("%d", i);
  i++;
}
```

Use When:

Number of iterations is not known

2. for Loop

Used when number of iterations is known.

Syntax

```
for(initialization; condition; increment)
{
  statements;
}
```

Example

```
for(int i = 1; i <= 5; i++)
{
  printf("%d", i);
}
```

Use When:

Fixed number of times

3. do-while Loop

Executes at least once.

Syntax

```
do
{
  statements;
}
while(condition);
```

Example

```
int i = 1;
do
{
  printf("%d", i);
  i++;
}
while(i <= 5);
```

Use When:

Menu programs or input validation

Comparison

Loop	Condition	Runs
while	Before	0 or more times
for	Before	0 or more times
do-while	After	At least once

Q7 Explain the use of break, continue, and goto statements in C. Provide examples of each.

Jump statements are used to control the flow of program.

Types:

break

continue

goto

1. break Statement

The break statement is used to exit from loop or switch immediately.

Example

```
#include <stdio.h>
int main()
{
   int i;

   for(i = 1; i <= 5; i++)
   {
      if(i == 3)
         break;

      printf("%d\n", i);
   }

   return 0;
}
```

Output

1

2

Use: Stop loop when condition is met

2. continue Statement

The continue statement skips current iteration and moves to next.

Example

```
#include <stdio.h>
int main()
{
   int i;

   for(i = 1; i <= 5; i++)
   {
      if(i == 3)
         continue;

      printf("%d\n", i);
   }

   return 0;
}
```

Output

1

2

4

5

Use: Skip specific condition

3. goto Statement

The goto statement transfers control to a labeled statement.

Example

```
#include <stdio.h>
int main()
{
   int i = 1;

start:
   printf("%d\n", i);
   i++;

   if(i <= 5)
      goto start;

   return 0;
}
```

Output

1

2

3

4

5

Q8 What are functions in C? Explain function declaration, definition, and how to

call a function. Provide examples.

Functions are blocks of code used to perform a specific task.

They make program easy, reusable, and organized.

Parts of Function

Function Declaration

Function Definition

Function Call

1. Function Declaration

Tells function name and parameters.

```
int add(int, int);
```

2. Function Definition

Contains actual code.

```
int add(int a, int b)
{
  return a + b;
}
```

3. Function Call

Used to execute function.

```
sum = add(10, 20);
```

Example Program

```
#include <stdio.h>

int add(int, int);   // Declaration
```

```
int main()
{
  int sum;
  sum = add(10,20);   // Function Call
  printf("%d", sum);
  return 0;
}

int add(int a, int b)   // Definition
{
  return a + b;
}
```

Output

30

Q9 Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

An array is a collection of same data type elements stored in continuous memory.

Example:

```
int a[5] = {10,20,30,40,50};
```

1. One-Dimensional Array

Stores elements in single row.

Example

```
#include <stdio.h>
int main()
{
  int a[5] = {1,2,3,4,5};
  for(int i=0;i<5;i++)
  {
    printf("%d ",a[i]);
  }
  return 0;
}
```

2. Multi-Dimensional Array

Stores elements in rows and columns.

Example

```
#include <stdio.h>
int main()
{
  int a[2][2] = {{1,2},{3,4}};
  for(int i=0;i<2;i++)
  {
    for(int j=0;j<2;j++)
    {
      printf("%d ",a[i][j]);
    }
    printf("\n");
  }
  return 0;
}
```

Difference

One-Dimensional Multi-Dimensional

Single row	Rows and columns
One index	Two indexes
Example: a[5]	Example: a[2][2]

Q10 Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

A pointer is a variable that stores the address of another variable.

Example:

int a = 10;

int *p = &a;

a → normal variable

*p → pointer variable

&a → address of variable

Declaration of Pointer

Syntax

data_type *pointer_name;

Example

int *p;

Initialization of Pointer

Pointer is initialized using address operator (&)

Example

int a = 10;

int *p;

```
p = &a;
```

Or directly:

```
int a = 10;
int *p = &a;
```

Example Program

```
#include <stdio.h>
int main()
{
   int a = 10;
   int *p = &a;

   printf("Value of a = %d\n", a);
   printf("Address of a = %p\n", &a);
   printf("Value using pointer = %d", *p);

   return 0;
}
```

Output (Example)

```
Value of a = 10
Address of a = 6422296
Value using pointer = 10
```

Why Pointers are Important

Used for memory management

Used in arrays and functions

Used for dynamic memory allocation

Makes program faster and efficient

Q11 Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.

String handling functions are used to manipulate strings in C.

These functions are available in string.h header file.

```
#include <string.h>
```

1. strlen() – String Length

strlen() is used to find length of string.

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str[] = "Hello";
   printf("Length = %d", strlen(str));
   return 0;
}
```

Use: Find number of characters in string

2. strcpy() – Copy String

strcpy() is used to copy one string into another.

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str1[20] = "Hello";
   char str2[20];

   strcpy(str2, str1);

   printf("%s", str2);
   return 0;
}
```

Use: Copy username, password, text etc.

3. strcat() – Concatenate String

strcat() is used to join two strings.

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str1[20] = "Hello ";
   char str2[] = "World";

   strcat(str1, str2);
```

```
    printf("%s", str1);
    return 0;
}
```

Use: Combine first name and last name

4. strcmp() – Compare Strings

strcmp() is used to compare two strings.

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[] = "Hello";
    char str2[] = "Hello";

    if(strcmp(str1, str2) == 0)
        printf("Strings are equal");
    else
        printf("Strings are not equal");

    return 0;
}
```

Use: Login system, password check

5. strchr() – Find Character

strchr() is used to find character in string.

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
   char str[] = "Hello";

   printf("%s", strchr(str, 'l'));

   return 0;
}
```

Use: Search character in string

Summary

Function	Use
strlen()	Find length
strcpy()	Copy string
strcat()	Join strings
strcmp()	Compare strings
strchr()	Find character

Q12 Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

A structure is a user-defined data type used to store different types of data together.

Example: storing student details like name, age, and marks.

1. Structure Declaration

Structure is declared using struct keyword.

Syntax

```
struct structure_name
{
  data_type member1;
  data_type member2;
};
```

Example

```
struct student
{
  int roll;
  char name[20];
  float marks;
};
```

2. Structure Initialization

Structure variables are created after declaration.

Example

```
struct student s1 = {1, "Parth", 85.5};
```

3. Accessing Structure Members

Structure members are accessed using dot (.) operator

Example

```
printf("%d", s1.roll);
printf("%s", s1.name);
printf("%f", s1.marks);
```

Complete Example

```
#include <stdio.h>

struct student
{
  int roll;
  char name[20];
  float marks;
};

int main()
{
  struct student s1 = {1, "Parth", 85.5};

  printf("Roll = %d\n", s1.roll);
  printf("Name = %s\n", s1.name);
  printf("Marks = %.2f", s1.marks);

  return 0;
}
```

Output

```
Roll = 1
Name = Parth
Marks = 85.50
```

Q13 Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files

File handling is used to store data permanently in files instead of temporary memory.

It helps in saving, reading, and updating data.

Importance of File Handling

Store data permanently

Read data anytime

Handle large data

Useful for records (student, employee, etc.)

File Operations in C

Opening a File

Writing to File

Reading from File

Closing a File

1. Opening a File

Files are opened using fopen() function.

Syntax

```
FILE *fp;
fp = fopen("file.txt","mode");
```

Modes

Mode	Meaning
r	Read
w	Write
a	Append

Example

```
FILE *fp;
fp = fopen("data.txt","w");
```

2. Writing to File

Use fprintf() to write data.

Example

```
#include <stdio.h>

int main()
{
  FILE *fp;
  fp = fopen("data.txt","w");

  fprintf(fp,"Hello World");

  fclose(fp);
  return 0;
}
```

3. Reading from File

Use fscanf() to read data.

Example

```
#include <stdio.h>

int main()
{
  FILE *fp;
  char text[50];
```

```
  fp = fopen("data.txt","r");

  fscanf(fp,"%s",text);

  printf("%s",text);

  fclose(fp);
  return 0;
}
```

4. Closing a File

Use fclose() to close file.

```
fclose(fp);
```

Closing file is important to save data properly.

Summary

Function	Use
fopen()	Open file
fprintf()	Write file
fscanf()	Read file
fclose()	Close file